POSTECH

TECH CHALLENGE

Sistema Integrado de Atendimento e Execução de Serviços — Microsserviços, Saga Pattern e Automação de Build, Testes e Deploy

Projeto:	Sistema de Oficina Mecânica
Fase:	Fase 4 — Microsserviços, Saga e CI/CD (SOAT)
Participante:	Lucaz Gabriel Teixeira Denadai — RM37376
Arquitetura:	Microsserviços (OS, Billing e Operações) + Saga orquestrada
Data de Entrega:	03/11/2026
Backend:	.NET 10 / C#
Bancos de Dados:	PostgreSQL 16 (SQL) e DynamoDB (NoSQL)

© POSTECH — Programa de Especialização em Software Architecture · Tech Challenge — SOAT — Fase 4

Descrição da solução e objetivos da fase

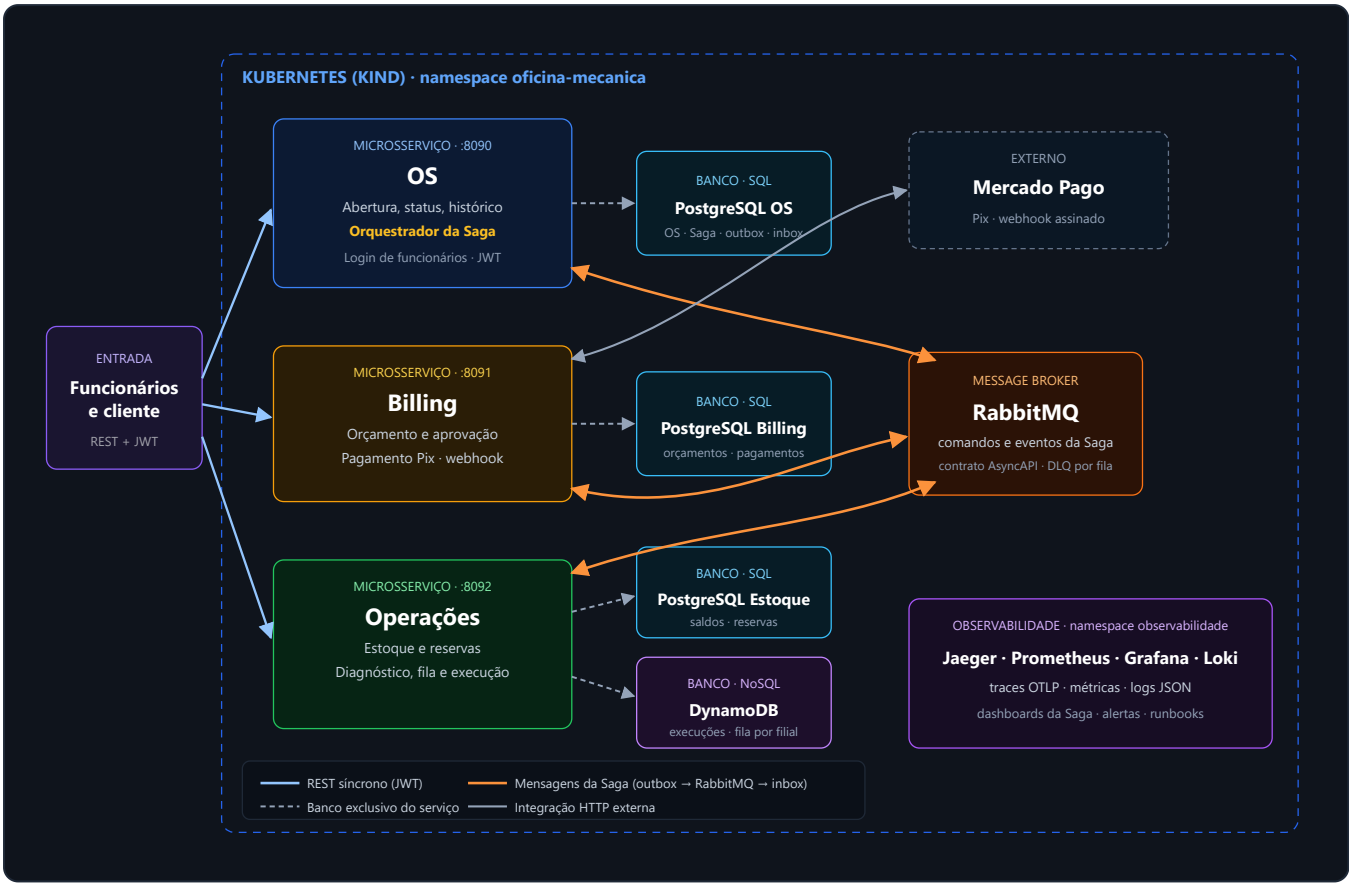
A oficina mecânica passou a operar em escala nacional, com várias filiais. Na Fase 3, Atendimento e Estoque rodavam na AWS. Nesta Fase 4, a aplicação foi refatorada para **três microsserviços independentes**, cada um com repositório, banco de dados e pipeline próprios: **OS** (ordens de serviço), **Billing** (orçamento e pagamento) e **Operações** (estoque e execução).

Abrir uma OS, aprovar o orçamento, pagar e executar o reparo agora atravessa três serviços, e cada um grava no próprio banco. Uma transação única não é mais possível. A consistência vem do **Saga Pattern**: cada etapa confirma o próprio efeito, e uma falha posterior dispara compensações que desfazem o que já foi feito (liberar a reserva de estoque, estornar o pagamento).

Objetivos alcançados nesta fase

- Três microsserviços com bancos próprios: PostgreSQL no OS, no Billing e no estoque de Operações; DynamoDB na execução de Operações. Nenhum serviço acessa o banco de outro.
- Comunicação assíncrona por RabbitMQ com contrato AsyncAPI, outbox transacional e inbox para deduplicação; REST para as ações de usuários.
- Saga **orquestrada** pelo serviço OS, com compensações, prazos com reenvio e resolução manual pelo Admin.
- Pagamento Pix pela API de Pagamentos do Mercado Pago, com webhook assinado e reconciliação.
- Testes unitários e de integração (Testcontainers) nos três serviços, com cobertura de linhas e de branches ≥80% exigida no CI, e o fluxo completo testado em BDD (Gherkin, Reqnroll).
- Pipeline independente por serviço: build, testes, gate de cobertura, qualidade (SonarCloud), imagem por commit e deploy automatizado em Kubernetes, com a `main` protegida.
- Observabilidade da Fase 3 reaproveitada: traces, métricas, logs, dashboards e alertas do fluxo distribuído.

Diagrama geral da arquitetura



Cada serviço tem banco exclusivo e conversa com os outros só por mensagens no RabbitMQ. Os três serviços exportam traces, métricas e logs para a stack de observabilidade. No CI de cada serviço, o mesmo Kind sobe só com a plataforma e aquele serviço.

Divisão dos microsserviços e bancos

Serviço	Responsabilidade	Banco	Por que é um serviço próprio
OS	Abertura, status e histórico da OS; cadastro de clientes, veículos e filiais; login de funcionários; orquestra a Saga.	PostgreSQL	A OS é o processo de negócio que atravessa as áreas: quem coordena a Saga precisa ser dono do estado dela.
Billing	Orçamento a partir do diagnóstico, aprovação pelo cliente, cobrança Pix, webhook, reconciliação e estorno.	PostgreSQL	Integra um provedor externo de pagamento, com regras e falhas próprias (webhook, reconciliação), e precisa de transação ACID para dinheiro.
Operações	Catálogo, estoque por filial e reservas; diagnóstico, fila e execução do reparo.	PostgreSQL (estoque) e DynamoDB (execução)	Estoque e execução são o trabalho da oficina: escalam com o volume de reparos, não com o de pagamentos.

Por que SQL e NoSQL: estoque e pagamentos precisam de consistência forte (reserva e baixa de saldo, valores), então ficam em PostgreSQL. A execução é um documento por OS, consultado por chave e pela fila

de cada filial, sem junções: um modelo chave-valor no DynamoDB atende e cumpre o NoSQL exigido (ADR-016). Na demonstração, o DynamoDB roda como DynamoDB Local no cluster.

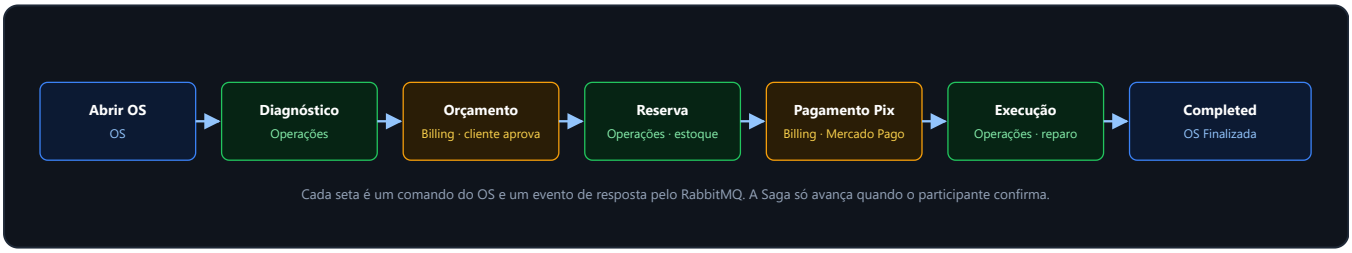
Saga Pattern: estratégia escolhida

Orquestração, com o serviço OS como orquestrador (ADR-017). Para cada OS existe uma Saga no banco do OS. A Saga guarda a etapa atual e o que já foi confirmado (reserva, pagamento, execução). Cada evento recebido muda a etapa e gera os comandos da próxima, gravados na outbox na mesma transação.

Por que orquestração e não coreografia

- **Um lugar só sabe o estado da OS.** Com coreografia, a pergunta "em que etapa está esta OS e o que falta desfazer?" exigiria juntar eventos de três serviços.
- **Compensações explícitas.** O fluxo tem compensações que dependem do que já aconteceu: liberar a reserva, estornar o pagamento, cancelar uma execução ainda na fila. No orquestrador, isso é uma regra só, testável.
- **Prazos e ação manual.** Sem resposta de um participante, a Saga reenvia o comando (mesma `idempotencyKey`) e, esgotados os reenvios, para em ação manual para o Admin decidir Cancelar ou Retomar (ADR-020).
- **Custo aceito:** o OS conhece a sequência das etapas. Os participantes continuam donos das próprias regras e só respondem a comandos.

Fluxo normal



Falhas e compensações

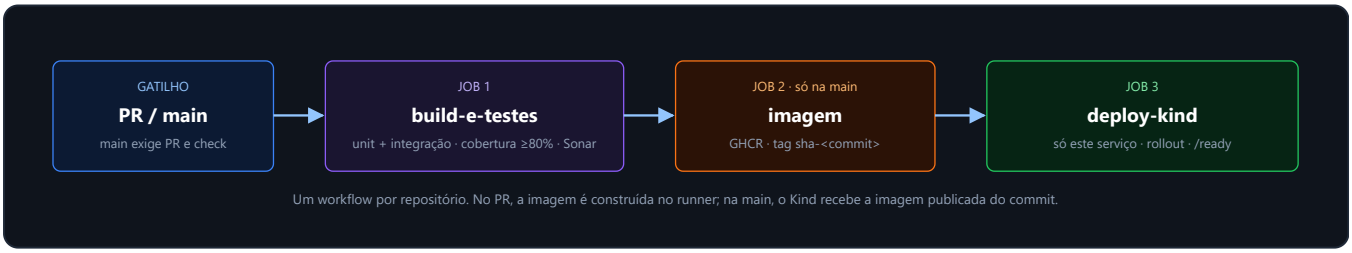
Falha	O que a Saga faz	Estado final
Cliente recusa o orçamento, ou o orçamento expira	Encerra sem cobrar nem reservar.	`Cancelled`
Falta estoque na filial	Encerra antes de cobrar.	`Cancelled`
Pagamento recusado ou expirado	Libera a reserva de estoque.	`Cancelled`
Reparo falha depois do pagamento	Estorna o pagamento e libera a reserva.	`Compensated`
Cancelamento pedido com a OS na fila	Pede o cancelamento a Operações; se aceito, estorna e libera.	`Compensated` ou `Cancelled`
Participante não responde	Reenvia o comando a cada prazo; depois de 3 reenvios, para para o Admin.	`ManualActionRequired`
Efeito chega depois do encerramento (pagamento ou reserva atrasados)	Desfaz o efeito: estorna ou libera.	mantém o final

Garantias de mensageria: outbox na mesma transação do dado (nenhum evento se perde se o broker cair), inbox por `messageId` (mensagem repetida não aplica duas vezes), `idempotencyKey` por etapa nos comandos e DLQ com o motivo para mensagens fora do contrato. As outboxes são seguras com mais de uma réplica (`FOR UPDATE SKIP LOCKED` no PostgreSQL e reserva condicional no DynamoDB).

Tecnologias utilizadas

Área	Tecnologia	Uso
Serviços	.NET 10, ASP.NET Core, Arquitetura Hexagonal	APIs REST com Swagger; domínio sem dependências externas
Bancos	PostgreSQL 16 (EF Core), DynamoDB (AWS SDK; DynamoDB Local no cluster)	Um banco por serviço; SQL e NoSQL
Mensageria	RabbitMQ 3.13, AsyncAPI 3.0	Comandos e eventos da Saga, contrato versionado no `tech-challenge-docs`
Pagamento	Mercado Pago (API de Pagamentos, Pix)	Cobrança, webhook com assinatura, reconciliação e estorno; provedor simulado na demonstração
Segurança	JWT (login do OS), roles Admin, Atendente, Mecânico e Cliente	Mesmo token aceito nos três serviços; token de cliente no formato da Lambda de CPF
Testes	xUnit, Testcontainers, Reqnroll (BDD), Coverlet, ReportGenerator	Unitários e de integração por serviço; fluxo completo em Gherkin
CI/CD	GitHub Actions, GHCR, SonarCloud, Kind, Kustomize	Pipeline por serviço com gate de cobertura e deploy automatizado
Observabilidade	OpenTelemetry, Jaeger, Prometheus, Grafana, Loki/Promtail	Traces entre serviços, métricas da Saga, logs por `correlationId`, alertas

CI/CD e deploy automatizado



Cada serviço implanta só a si mesmo: o job sobe um Kind no runner com a plataforma compartilhada (RabbitMQ, Jaeger) do `tech-challenge-infra-k8s`, aplica os manifests do próprio repositório e falha se o rollout ou o readiness não ficarem prontos. No OS, um workflow separado roda o BDD com os três serviços de verdade.

Monitoramento e rastreamento

As ferramentas da Fase 3 — Jaeger, Prometheus, Grafana e Loki — rodam no Kind, sem custo (ADR-022). O `traceparent` vai na outbox e no cabeçalho da mensagem, então o trace continua entre os serviços. Os logs JSON levam o `CorrelationId` da Saga e o `TraceId`. O dashboard **Saga da OS** mostra Sagas por estado, duração, etapa mais antiga, reenvios, ações manuais e DLQs. Seis alertas têm runbook: Saga em ação manual, pagamento pendente, Saga parada, DLQ com mensagens, deploy indisponível e serviço fora do ar.

Principais decisões arquiteturais

Onde passam as fronteiras dos serviços

Os exemplos do enunciado foram seguidos, com um ajuste: Estoque não virou um quarto serviço. A reserva e a baixa de peças acontecem dentro da execução do reparo, então ficaram em Operações com a execução (ADR-014). Isso evita uma Saga a mais entre "reservar" e "executar".

Por que a outbox e a inbox

Gravar no banco e publicar no RabbitMQ são duas operações que podem falhar separadamente. Com a outbox, o evento é gravado na mesma transação do dado e um despachante o publica depois. Se o broker cair, o evento espera. Do outro lado, a inbox registra cada `messageId`: a entrega "ao menos uma vez" do RabbitMQ não aplica o mesmo efeito duas vezes.

Pagamento: o resultado pode ser desconhecido

Um timeout na chamada ao Mercado Pago não quer dizer "recusado". O Billing trata como resultado desconhecido e reconcilia, consultando o provedor até saber, e o webhook assinado também traz o resultado. A Saga espera em `ReconcilingPayment` em vez de cancelar uma cobrança que pode ter sido paga (ADR-019).

Kubernetes sem AWS nesta fase

Sem conta AWS disponível na Fase 4, o deploy automatizado roda em Kind: um cluster efêmero no CI de cada serviço e o mesmo cluster na demonstração (ADR-021). Os manifests de cada serviço não dependem do provedor; trocar o Kind por um cluster gerenciado muda só a plataforma. A Lambda de autenticação por CPF não roda nesta fase; o token de cliente é gerado no mesmo formato para a demonstração.

Qualidade como condição de merge

A `main` dos três serviços exige PR e o check verde, também para administradores. O gate mede linhas **e** branches (≥80% nos dois), porque linhas sozinhas deixavam caminhos de erro sem teste. O SonarCloud roda quando os segredos do projeto estão configurados.

> **Todos os ADRs, com contexto, alternativas consideradas e consequências**
> https://github.com/LucazDenadai/tech-challenge-docs/tree/main/adr

Instruções de execução

Demonstração completa no Kind

```
git clone https://github.com/LucazDenadai/tech-challenge-infra-k8s.git
git clone https://github.com/LucazDenadai/tech-challenge-os.git
git clone https://github.com/LucazDenadai/tech-challenge-billing.git
git clone https://github.com/LucazDenadai/tech-challenge-operacoes.git
cd tech-challenge-infra-k8s && ./kind/subir.sh
```

O script cria o cluster, gera os segredos, sobe RabbitMQ, Jaeger, Prometheus, Grafana e Loki, constrói e implanta os três serviços e imprime as URLs e as senhas geradas. Requisitos: Docker, kind e kubectl.

Serviço	Endereço
OS · Billing · Operações (Swagger)	`localhost:8090/os/swagger` · `localhost:8091/billing/swagger` · `localhost:8092/operacoes/swagger`
Grafana · Prometheus · Jaeger · RabbitMQ	`localhost:13000` · `localhost:19090` · `localhost:16690` · `localhost:15680`

Testes de cada serviço

```
dotnet test     # na raiz do repositório; a integração usa Testcontainers (Docker)
```

Instruções completas, cobertura e arquitetura de cada serviço estão no README de cada repositório.

Links da entrega

Serviço OS (orquestrador da Saga)
https://github.com/LucazDenadai/tech-challenge-os

Serviço Billing (orçamento e pagamento)
https://github.com/LucazDenadai/tech-challenge-billing

Serviço Operações (estoque e execução)
https://github.com/LucazDenadai/tech-challenge-operacoes

Plataforma Kubernetes e observabilidade
https://github.com/LucazDenadai/tech-challenge-infra-k8s

Documentação: ADRs, contrato AsyncAPI, diagramas e cards
https://github.com/LucazDenadai/tech-challenge-docs

Collection Postman da Saga nos três serviços
https://github.com/LucazDenadai/tech-challenge-docs/tree/main/postman

Vídeo demonstrativo (YouTube)
https://youtu.be/l4jkgqyC1sA
Demonstra o fluxo completo de uma OS pelos três serviços, a Saga com falha e compensação, o deploy automatizado com validação de testes e o monitoramento e rastreamento do fluxo distribuído.

© POSTECH — Programa de Especialização em Software Architecture